ISAE-ENSMA **2023-2024**
Master

Examen Informatique Décisionnelle et Big Data
Documents de cours et de TD sont autorisés

Exercice 1 : Fragmentation mixte

Nous avons vu en cours qu'une table relationnelle peut être partitionnée en utilisant la fragmentation mixte. La fragmentation mixte résulte de l'application successive de la fragmentation horizontale (ou verticale) et verticale (ou horizontale) sur une table.
Question 1 : L'ordre de l'application de la fragmentation a-t-il une importance sur (a) le nombre de fragments générés et (b) la performance de requêtes. Justifiez votre réponse par un exemple.
Question 2 : Formalisez le problème de la fragmentation mixte comme un problème d'optimisation avec une **contrainte à déterminer**.
Question 3 : Proposez une heuristique pour la fragmentation mixte.
Question 4 : Donnez la complexité de cet algorithme.

Exercice 2 : Évaluation d'une Requête Répartie

Considérer une base de données de deux tables ayant le schéma suivant :

R(Enum, Name, Age, Rank)
S(Enum, Jnum, Resp, Dur)
fragmenté en plusieurs fragments :

- $S1 = \sigma_{dur < 12}(S)$
- $S2 = \sigma_{dur \geq 12}(S)$
- $R1 = \pi_{enum, name, age}(R \text{ join } S1)$
- $R2 = \pi_{enum, rank}(R \text{ join } S1)$
- $R3 = \pi_{enum, name, age}(R \text{ join } S2)$
- $R4 = \pi_{enum, rank}(R \text{ join } S2)$

On supposera que tout *enum* de R est aussi présent comme *enum* de S et l'optimisation doit utiliser cette information.
Question 5 : Peut-on reconstruire la base de données à partir de ces fragments ? Comment ?
Question 6 : Cette fragmentation est-elle disjointe ? Expliquer.
Question 7 : Transformer la requête suivante en une requête sur les fragments :

```
SELECT Name
FROM R, S
WHERE (R.Enum = S.Enum AND Dur = 30)
OR (R.Enum = S.Enum AND Age = 55)
```

Question 8 : Optimiser cette requête.

Exercice 3 : Optimisation basée sur modèle de coût

Soit une base de données composée de deux relations Fournisseur et Livraison :

Fournisseur(NF, NomF, Adresse)
Livraison(NP, NF, Date, Quantité)

Soit la requête Q à optimiser qui consiste à lister les noms des fournisseurs qui livrent le produit dont le numéro p1.

Question 9 : Exprimer cette requête en algèbre relationnelle et en SQL.
On dispose des informations suivantes pour optimiser la requête :

- La relation *Fournisseur* comporte 10 0000 tuples ;
- La relation *Livraison* comporte 10 000 tuples dont une centaine concernant le produit P1 ;
- Le *facteur de blocage* pour les deux relations est 5 (chaque bloc contient 5 tuples) ;
- Les deux tables sont tassées ;
- Une entrée-sortie disque prend 20 ms par bloc ;
- Il est possible de stocker jusqu'à une centaine de tuples en mémoire centrale.
- Pour exécuter cette requête, plusieurs plans sont envisagés :

Plan 1 : On effectue naïvement la jointure, suivie de la sélection : l'algorithme est le suivant :

```
tant que non fin Fournisseur faire          10 000 x 10 000
        lire F de Fournisseur
        pour tout tuple f de F faire
                tant que non fin Livraison faire
                        lire L de Livraison
                        pour tout tuple l de L faire
                                si (f.NF = l.NF et l.NP = "p1") alors
                                        produire le tuple l.NOMF
                                fsi
```

Plan 2 : On exploite le fait qu'il est possible de stocker 100 tuples en mémoire centrale :

```
Tant que non fin fournisseur ~~100~~ 100x
        lire 100 tuples de Fournisseur 20
        tant que non fin Livraison faire
                lire L de Livraison 10000
                pour tout f dans les 100 tuples de Fournisseur faire
                        pour tout l de L faire
                                si (f.NF = l.NF et l.NP = "p1") alors
                                        produire le tuple l.NOMF
                                fsi
```

Plan 3 : La requête est optimisée par une évaluation aussitôt que possible de la sélection, ce qui donne l'algorithme suivant :

```
tant que non fin Livraison faire
        lire L de Livraison 10 000          100  to gagne
        pour tout l de L faire
                si l.NP = "p1" alors
                        insérer l dans L1   10 000 → 2000
                fsi
tant que non fin Fournisseur faire 100000/5
50      lire 100 tuples de Fournisseur
        tant que non fin L1 faire 100x      50
                lire ~~L de~~ L1  100 50
                pour tout f dans les 100 tuples de Fournisseur faire
                        pour tout l de L1 faire     50
                                si (f.NF = l.NF) alors
                                        produire le tuple l.NOMF
                                fsi
```

Pour chacun des plans,

Question 10 : Combien de lecture de blocs sont effectués ?

Question 11 : Quel est le temps d'exécution de la requête ?

Question 12 : Proposez d'autres plans d'exécution ?

410 000

↓

402000.

2%

Exercice 4 : Index de jointure binaire

Soit un schéma d'une base de données en étoile de trois tables de dimensions (Client, Produit et Temps) et une table des faits (Ventes) dont la population est donnée ci-dessous. Chaque tuple de table est associé à un RID. Sur ce schéma la requête Q suivante est exécutée :

```
SELECT Count(*)
FROM Client C, Produit P, Temps T, Ventes V
WHERE C.City ='Poitiers' AND  P.Type= 'Beauté'
AND T.Mois = 'Juin' AND P.PID = V.PID
AND C.CID = V.CID AND T.TID = V.TID
```

Question 13 : Proposer très scénarii d'optimisation de la requête en utilisant :

- **Scénario 1** : un seul index de jointure binaire
- **Scénario 2** : deux index de jointure binaire
- **Scénario 3** : trois index de jointure binaire

Question 14 : Montrer comment l'optimiseur de requêtes utilise ces index dans les trois scénarii pour traiter la requête **Q.**

Question 15 : Quel est le résultat retourné par cette requête ?

Question 16 : Est-ce que le résultat dépend des scénarii

Question 17 : Quel est le nombre de tuples de la table des faits que l'optimiseur a passé en revue (scanné) lors de l'exécution de la requête ?

CLIENT

C_RID	CID	Nom	Ville
6	616	Gilles	Poitiers
5	515	Yves	Paris
4	414	Yamine	Nantes
3	313	Ladjel	Nantes
2	212	Eric	Poitiers
1	111	Pascal	Poitiers

P_RID	PID	Nom	Type
6	106	Sonoflore	Beauté
5	105	Clarins	Beauté
4	104	WebCam	Multimédia
3	103	Barbie	Jouet
2	102	Engrais	Jardinage
1	101	SlimForm	Fitness

PRODUIT

T_RID	TID	Jour	Année
1	11	janvier	2003
2	22	février	2003
3	33	mars	2003
4	44	Avril	2003
5	55	Mai	2003
6	66	Juin	2003

TEMPS

VENTES

V_RID	CID	PID	TID	Ventes
1	616	106	11	25
2	616	106	66	28
3	616	104	33	50
4	515	104	11	10
5	414	105	66	14
6	212	106	55	14
7	111	101	44	20
8	111	101	33	27
9	212	101	11	100
10	313	102	11	200
11	414	102	11	102
12	414	102	55	203
13	515	102	66	100
14	515	103	55	17
15	212	103	44	45
16	111	105	66	44
17	212	104	66	40
18	515	104	22	20
19	616	104	22	20
20	616	104	55	20
21	212	105	11	10
22	212	105	44	10
23	212	105	55	18
24	212	106	11	18
25	313	105	66	19
26	313	105	22	17
27	313	106	11	15